

A COLLEGE DOMAIN SPECIFIC FIXMATE SERVICE FINDER

CLOUD COMPUTING

Submitted by

HARSHA VARDHINI(230701109)

PRATHEBA D (230701239)

DIVYA(230701082)

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING in COMPUTER SCIENCE AND ENGINEERING



**DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING**

RAJALAKSHMI ENGINEERING COLLEGE

MAY, 2026

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this project report titled “**FIXMATE SERVICE FINDER**” is the bonafide work of **HARSHA VARDHINI 230701109** , **PRATHEBA D 230701239**, **DIVYA 230701082** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. E. M Malathy
Professor &
Head of The Department
Department of Computer Science
and Engineering
Rajalakshmi Engineering College

SIGNATURE

Ponmani S
Supervisor
Associate Professor
Department of Computer Science
and Engineering
Rajalakshmi Engineering College

Submitted to Project Viva Voce Examination held on _____

Internal Examiner

External Examiner

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S.MEGANATHAN, B.E, F.I.E.**, our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.**, and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.**, for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.**, our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr.MALATHY E.M, Ph.D.**, Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, **DR. M. AYYADURAI M.E.,Ph.D.**, Associate Professor, Department of Computer Science and Engineering for his valuable guidance throughout the course of the project.

HARSHA VARDHINI
PRATHEBA D
DIVYA

ABSTRACT

In today's rapidly evolving digital landscape, the demand for quick, reliable, and accessible everyday services has increased significantly. Essential services such as plumbing, electrical repairs, cleaning, and appliance maintenance are integral to daily life, yet finding trustworthy service providers remains a major challenge for many individuals. Traditional methods of locating such services—through personal contacts, local advertisements, or unverified online sources—are often inefficient, time-consuming, and lack transparency. This creates inconvenience for users and limits opportunities for skilled service providers to reach a broader audience.

The FixMate project addresses these challenges by developing a centralized, user-friendly digital platform that connects users with verified service providers. The primary objective of the system is to simplify the process of discovering, booking, and managing essential services through an intuitive and efficient interface. By integrating multiple services into a single platform, FixMate eliminates the need for users to rely on fragmented and unreliable sources.

The platform is designed using a client-server architecture, where the frontend provides an interactive user interface and the backend manages business logic, data processing, and communication with the database. Users can register, log in securely, browse available service categories, and book services based on their requirements. Service providers can create profiles, list their offerings, and manage incoming service requests. The system also supports booking management and status updates, ensuring smooth communication between users and providers.

One of the key features of FixMate is its focus on reliability and scalability. The platform is built using modern web technologies, enabling efficient performance and the ability to handle increasing numbers of users and services. Security measures such as authentication systems are implemented to protect user data and ensure safe interactions. Additionally, the system is designed with flexibility in mind, allowing for future enhancements such as online payment integration, real-time tracking, and user feedback mechanisms.

The project also aims to create a balanced ecosystem that benefits both users and service providers. While users gain convenient access to essential services, service providers are given a digital platform to expand their reach and improve their visibility. This contributes to economic opportunities and promotes the adoption of digital solutions in everyday service management.

In conclusion, FixMate provides an effective solution to the problem of service accessibility by combining technology, usability, and scalability. It enhances user convenience, improves service reliability, and establishes a foundation for future advancements in the domain of on-demand service platforms.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1	Introduction	1
1.1	Problem Statement	2
1.2	Objective of the Project	4
1.3	Scope and Boundaries	6
1.4	Stakeholders and End Users	8
1.5	Technologies Used	10
1.6	Organization of the Report	12
2	System Design and Architecture	14
2.1	Requirement Summary (Functional & Non-Functional)	15
2.2	Proposed Solution Overview	17
2.3	Cloud Deployment Strategy	19
2.4	Infrastructure Requirements	21
2.5	Azure Services Mapping and Justification	23
3	DevOps Implementation	26
3.1	Continuous Integration and Deployment (CI/CD) Setup	27
3.2	Terraform Infrastructure-as-Code (IaC)	29
3.3	Containerization Strategy (Docker)	31
3.4	Kubernetes Orchestration (AKS Deployment, Scaling)	33
3.5	GenAI Integration and Azure AI Service Mapping	35
4	Cloud Operations and Security	38
4.1	DevSecOps Integration (CodeQL / SonarCloud / ZAP Scans)	39
4.2	Monitoring and Observability (Azure Monitor / App Insights)	41
4.3	Access Control (RBAC Overview)	43

4.4	Blue–Green Deployment & Disaster Recovery Planning	45
5	Results and Discussion	48
5.1	Implementation Summary	49
5.2	Challenges Faced and Resolutions	51
5.3	Performance or Cost Observations	53
5.4	Key Learnings and Team Contributions	55
6	Conclusion and Future Enhancement	58
6.1	Conclusion	59
6.2	Future Scope	61
	References	63
	Appendices	65

CHAPTER 1 - INTRODUCTION

1.1 PROBLEM STATEMENT

In today's fast-paced environment, individuals often struggle to find reliable and efficient service providers for routine household and personal needs. The absence of a unified platform for accessing multiple essential services leads to fragmented and inefficient solutions. Users are forced to depend on informal sources such as word-of-mouth recommendations, local advertisements, or unverified online listings, which may not guarantee quality or reliability.

- One of the primary challenges is the lack of trust and transparency in the service selection process. Users have limited information about the qualifications, experience, and credibility of service providers. This uncertainty often results in poor service experiences, financial losses, or even safety concerns. Additionally, there is no standardized mechanism for comparing services based on pricing, availability, or user feedback.
- Another major issue is the inefficiency of the booking and communication process. Traditional methods involve multiple phone calls, scheduling conflicts, and unclear timelines, which can be frustrating and time-consuming. There is also no proper system for tracking service requests, leading to confusion and lack of accountability.
- From the perspective of service providers, there is limited visibility and reach. Skilled workers often struggle to connect with potential customers due to the absence of a digital platform that showcases their services. This results in underutilization of resources and missed opportunities for income generation.
- Moreover, the lack of integration between users and service providers creates a gap in the service ecosystem. There is no centralized system to manage interactions, bookings, and service history, which further complicates the process.
- Therefore, the problem lies in the absence of a reliable, efficient, and user-friendly platform that can seamlessly connect users with verified service providers while ensuring transparency, convenience, and quality assurance.

1.2 OBJECTIVE OF THE PROJECT

The primary objective of the FixMate project is to design and develop a comprehensive digital platform that simplifies the process of accessing essential services. The system aims to provide a seamless and efficient interface where users can easily discover, book, and manage various household and personal services without unnecessary complexity.

- One of the key objectives is to centralize multiple services into a single platform. By doing so, users no longer need to rely on multiple sources to find different types of service providers. Instead, they can access a wide range of services in one place, saving time and effort. This centralization also enhances user convenience and improves the overall experience.
- Another important objective is to ensure trust and reliability. The platform is designed to include mechanisms for verifying service providers, thereby increasing user confidence. In future implementations, features such as ratings, reviews, and feedback systems can further strengthen trust and accountability.
- The project also aims to streamline the booking process. Users should be able to schedule services with minimal effort, view availability, and receive confirmations instantly. This eliminates the inefficiencies associated with traditional booking methods and improves communication between users and providers.

- Additionally, the system seeks to empower service providers by offering them a digital platform to showcase their skills and reach a broader audience. This not only increases their visibility but also creates opportunities for economic growth and professional development.
- Scalability is another critical objective. The platform is designed in such a way that it can be expanded to include additional services, features, and users without significant changes to the core architecture.
- Ultimately, the FixMate project aims to create a reliable, efficient, and scalable ecosystem that enhances service accessibility while benefiting both users and service providers.

1.3 SCOPE AND BOUNDARIES

The scope of the FixMate project encompasses the development of a digital platform that facilitates the connection between users and service providers for essential services. The system is designed to support a variety of common household services, including plumbing, electrical repairs, cleaning, and appliance maintenance. These services are selected based on their high demand and relevance in everyday life.

Within the scope, the platform provides functionalities such as user registration and authentication, service browsing, booking management, and service provider interaction. Users can create accounts, explore available services, select providers, and schedule appointments. Service providers, on the other hand, can register their profiles, list their services, and manage incoming requests.

The system also includes basic administrative controls to manage users, services, and overall platform operations. This ensures that the platform remains organized and functional.

However, there are certain boundaries and limitations to the project. In its initial phase, the platform may support only a limited set of service categories. Advanced features such as real-time GPS tracking, AI-based recommendations, or dynamic pricing may not be included initially but can be added in future iterations.

Another limitation is the dependency on internet connectivity. Since the platform is web-based, users must have access to a stable internet connection to utilize its features. Additionally, the quality of services ultimately depends on the individual service providers, which may vary despite platform efforts to ensure reliability.

Payment integration may also be basic or optional in early versions, with more advanced and secure payment systems planned for future development.

Overall, while the scope of FixMate is focused on delivering a functional and efficient service platform, its boundaries are defined by implementation constraints and the need for gradual enhancement.

1.4 STAKEHOLDERS AND END USER

The FixMate project involves multiple stakeholders and end users, each playing a crucial role in the system's functionality and success. Understanding these groups is essential for designing a platform that meets diverse needs and expectations.

- The primary stakeholders include the development team, who are responsible for designing, implementing, and maintaining the system. Their role involves ensuring that the platform is functional, secure, and scalable. Academic evaluators or project supervisors may also be considered stakeholders, as they assess the quality and effectiveness of the project.
- Service providers form another key group of stakeholders. These individuals or businesses offer various services through the platform. Their participation is vital for the system's success, as they directly interact with users and deliver the services. The platform must cater to their needs by providing tools for managing bookings, updating availability, and communicating with customers.
- Platform administrators are also important stakeholders. They oversee the overall operation of the system, manage user accounts, monitor activity, and ensure compliance with policies. Their role is crucial in maintaining system integrity and resolving issues.
- The end users primarily consist of individuals seeking services. These may include homeowners, tenants, working professionals, or small business owners who require quick and reliable solutions for everyday needs. The platform is designed to provide them with a convenient and efficient way to access services.

Additionally, potential investors or organizations interested in scaling the platform can be considered secondary stakeholders. Their involvement can support future growth and expansion.

1.5 TECHNOLOGIES USED

The FixMate project utilizes a combination of modern web development technologies to ensure scalability, performance, and a smooth user experience. The key technologies used in the system are as follows:

Frontend Technologies

HTML (HyperText Markup Language)

- Used to structure the content of web pages
- Defines elements such as forms, buttons, and layouts

CSS (Cascading Style Sheets)

- Responsible for styling and visual presentation
- Ensures responsive design for different screen sizes

JavaScript

- Adds interactivity and dynamic behavior to the application
- Handles user actions such as clicks, form submissions, and validations

Frontend Framework (if applicable – React/Angular)

- Enables component-based architecture
- Improves code reusability and maintainability
- Enhances performance with efficient rendering

Backend Technologies

Node.js

- Provides a runtime environment for executing JavaScript on the server
- Enables fast and scalable backend development

Express.js

- Lightweight web framework for Node.js
- Used to build RESTful APIs and handle routing
- Simplifies server-side logic implementation

Database Technologies

MongoDB (NoSQL Database)

- Stores data in flexible, JSON-like format
- Suitable for handling dynamic and unstructured data

Version Control

Git

- Tracks changes in source code
- Enables collaboration among team members

GitHub

- Cloud-based repository hosting
- Facilitates code sharing, version management, and teamwork

Deployment & Hosting

Amazon Web Services

- Used for deploying and hosting the application
- Provides scalability and high availability

Additional Tools & Technologies

RESTful APIs

- Enables communication between frontend and backend
- Ensures structured data exchange

Authentication Systems

- Secures user login and registration processes
- Protects user data and access

UI/UX Design Tools

- Used for designing user-friendly interfaces
- Improves overall user experience

1.6 ORGANIZATION OF THE REPORT

This report is structured to provide a complete and logical documentation of the entire project lifecycle.

Chapter 1 introduces the problem context, project objectives, scope, stakeholders, technologies, and report structure.

Chapter 2 presents a detailed examination of the system design and architecture, including requirements, solution overview, cloud strategy, infrastructure specifications, and service mappings.

Chapter 3 delves into the DevOps implementation, covering CI/CD pipelines, Terraform automation, Docker containerization, orchestration principles, and AI service integration. **Chapter 4** focuses on operational excellence and security, discussing DevSecOps practices, monitoring, access control, and deployment strategies.

Chapter 5 presents the results of the implementation, including summaries, challenges overcome, performance metrics, cost analysis, and key learnings.

Chapter 6 concludes the report with a summary of achievements and a roadmap for future enhancements.

The document concludes with references and appendices containing configuration details and system screenshots.

CHAPTER 2 - SYSTEM DESIGN AND ARCHITECTURE

2.1 REQUIREMENT SUMMARY

Functional Requirements

These describe the core functionalities of the system:

- **User Registration**
 - Users must be able to create an account
 - Requires input such as name, email, and password
- **User Login and Authentication**
 - Secure login system for registered users
 - Validates credentials before granting access
- **Service Browsing**
 - Users can view available service categories
 - Includes details such as type of service and description
- **Service Booking**
 - Users can select and book a service
 - Allows scheduling based on availability
- **Service Provider Registration**
 - Providers can create profiles and list services
 - Includes skills, experience, and service details
- **Booking Management**
 - Providers can accept or reject service requests
 - Users can view booking status
- **Notifications and Updates**
 - Users receive updates on booking status
 - Providers are notified of new requests
- **User Profile Management**
 - Users can update personal details
 - Allows viewing of booking history

Non-Functional Requirements

These define system performance and quality attributes:

- **Usability**
 - Interface must be simple and user-friendly
 - Easy navigation for all types of users
- **Performance**

- Fast response time for user actions
 - Efficient handling of multiple requests
- **Scalability**
 - System should support increasing users and services
 - Ability to expand without major redesign
- **Security**
 - Protection of user data through authentication
 - Prevention of unauthorized access
- **Reliability**
 - System should function consistently without failures
 - Minimal downtime during operation
- **Availability**
 - Platform should be accessible at all times
 - Ensures continuous service for users
- **Maintainability**
 - Code should be easy to update and manage
 - Supports future enhancements
- **Compatibility**
 - Works across different devices and browsers
 - Responsive design for mobile and desktop

2.2 PROPOSED SOLUTION OVERVIEWS

The FixMate platform is designed using a client-server architecture that separates the system into distinct components, each responsible for specific functionalities. This modular approach enhances scalability, maintainability, and overall system performance.

The frontend serves as the user interface, allowing users to interact with the platform. It provides features such as service browsing, booking, and account management. The design focuses on simplicity and responsiveness to ensure a smooth user experience across different devices.

The backend acts as the core of the system, handling business logic and processing user requests. It manages communication between the frontend and the database through RESTful APIs. This ensures efficient data exchange and maintains system integrity.

The database is responsible for storing all relevant information, including user profiles, service details, and booking records. It plays a crucial role in maintaining data consistency and supporting system operations.

The workflow begins when a user logs into the platform and selects a service. The request is sent to the backend, which processes the information and updates the database. The service provider is notified and can respond to the request. The status of the booking is then updated and displayed to the user.

The architecture follows principles such as separation of concerns and modular design, making it easier to expand and maintain. Future enhancements can be integrated without significant changes to the existing system.

Overall, the proposed solution provides a robust and efficient framework for delivering accessible and reliable services through the FixMate platform.

2.3 CLOUD DEPLOYMENT STRATEGY

1. Overview

The FixMate application follows a modern cloud deployment strategy by distributing its components across different cloud platforms. The frontend of the application is deployed on Vercel, while the backend is hosted on an Amazon Web Services (AWS) EC2 instance. This separation of concerns allows each part of the system to operate independently while still communicating efficiently through API requests.

This approach improves overall system performance, scalability, and maintainability. By using cloud services, the application avoids the limitations of local hosting and ensures that users can access the platform from anywhere with minimal latency. The architecture also supports future expansion, making it suitable for real-world production environments.

2. Backend Deployment (AWS EC2)

The backend of FixMate is deployed on an AWS EC2 instance, which acts as a virtual server in the cloud. An Ubuntu operating system is used due to its stability and compatibility with Node.js applications. After launching the EC2 instance, it is configured with necessary security rules to allow SSH access (port 22) and HTTP traffic (port 80).

The backend code is retrieved from a GitHub repository and installed on the server. All required dependencies are installed using npm, and environment-specific configurations such as database connection strings, API keys, and authentication secrets are stored securely in a `.env` file.

To ensure that the backend runs continuously without interruption, PM2 (Process Manager 2) is used. PM2 allows the application to run in the background as a daemon process. It automatically restarts the application in case of crashes and ensures that the server starts running again after a system reboot. This eliminates the need for manual intervention and guarantees high availability of the backend services.

3. Frontend Deployment (Vercel)

The frontend of the FixMate application is built using React and deployed on Vercel, a platform optimized for hosting modern web applications. Vercel provides seamless integration with GitHub, allowing automatic deployment whenever changes are pushed to the repository.

One of the key advantages of using Vercel is its global Content Delivery Network (CDN), which ensures that

the frontend is served quickly to users regardless of their geographical location. This significantly reduces loading times and enhances user experience.

The frontend is configured to communicate with the backend through API endpoints. These endpoints are mapped to the public IP address of the EC2 instance using a configuration file (such as `vercel.json`). This setup ensures that all client requests are correctly routed to the backend server for processing.

4. Networking and Integration

The backend application runs internally on port 5000 within the EC2 instance. However, standard web traffic typically uses port 80 for HTTP communication. To bridge this gap, port forwarding is implemented using iptables.

Port forwarding redirects all incoming traffic from port 80 to port 5000, allowing users to access the backend without specifying a custom port number in the URL. This simplifies the user experience and ensures compatibility with standard web practices.

The integration between frontend and backend is achieved through RESTful API calls. When a user performs an action (such as booking a service), the frontend sends a request to the backend, which processes the request, interacts with the database, and returns the appropriate response. This communication flow is essential for enabling dynamic features such as service booking, worker selection, tracking, and payment processing.

5. Reliability and Security

The deployment strategy incorporates several measures to ensure both reliability and security of the system. Reliability is achieved through the use of PM2, which keeps the backend running continuously and handles automatic restarts in case of failures. This ensures that the application remains available to users at all times.

Security is maintained through multiple layers. Access to the EC2 instance is restricted using SSH key-based authentication, preventing unauthorized login attempts. Security groups act as a firewall, allowing only specific ports and blocking unwanted traffic. Sensitive information such as API keys, database credentials, and authentication tokens are stored in environment variables instead of hardcoding them into the application.

Additionally, separating the frontend and backend reduces the attack surface, making the system more secure. The use of trusted cloud platforms like AWS and Vercel further enhances the overall reliability and safety of the deployment.

2.4 INFRASTRUCTURE REQUIREMENTS

1. Hardware Requirements

The FixMate application requires both development-side and cloud-side hardware resources to function efficiently.

For development, a standard system with at least 8 GB RAM, a multi-core processor, and stable internet connectivity is sufficient to run the frontend and backend locally.

For deployment, the application uses an AWS EC2 instance, which acts as a virtual server in the cloud. A basic instance type (such as t2.micro or t3.micro) is sufficient for initial deployment, providing adequate CPU, memory, and storage for handling moderate user traffic. As the number of users increases, the infrastructure can be scaled vertically (upgrading instance type) or horizontally (adding more instances).

2. Software Requirements

The system relies on a set of software tools and technologies for development and deployment:

- Operating System: Ubuntu (used in AWS EC2 for backend hosting)
- Runtime Environment: Node.js for executing backend code
- Package Manager: npm for managing dependencies
- Frontend Framework: React.js for building the user interface
- Backend Framework: Express.js for handling API requests
- Database: MongoDB (for storing user, worker, booking, and payment data)
- Version Control: Git and GitHub for source code management
- Deployment Platforms: AWS EC2 (backend) and Vercel (frontend)

These technologies work together to create a full-stack web application environment.

3. Cloud Infrastructure (AWS & Vercel)

The cloud infrastructure plays a critical role in hosting and delivering the application.

- AWS EC2: Used to host the backend server. It provides flexibility, scalability, and control over the server environment.
- Vercel: Used for deploying the frontend. It offers fast global delivery through a Content Delivery Network (CDN) and automatic deployment from GitHub.

This combination ensures efficient distribution of responsibilities, where the backend handles logic and data processing, while the frontend delivers a responsive user experience.

4. Networking Requirements

Proper networking configuration is essential for communication between different components of the system.

- Public IP Address: The EC2 instance is assigned a public IP to allow external access
- Open Ports:
 - Port 22 for SSH access
 - Port 80 for HTTP traffic
- Port Forwarding: Configured to redirect traffic from port 80 to port 5000, where the Node.js backend

runs

- API Communication: The frontend sends HTTP requests to the backend using REST APIs

These configurations ensure seamless interaction between users, frontend, and backend services.

5. Process Management and Reliability Tools

To maintain continuous operation of the backend, additional tools are used:

- PM2 (Process Manager):
 - Runs the backend in the background
 - Automatically restarts the application if it crashes
 - Ensures the app starts on server reboot
 - Provides monitoring and logging features

This guarantees high availability and reduces downtime without manual intervention.

6. Security Requirements

Security is an important aspect of the infrastructure setup.

- SSH Key Authentication: Secure access to the EC2 server using a private key (.pem file)
- Security Groups (Firewall): Restrict access to only required ports
- Environment Variables: Sensitive data such as API keys and database credentials are stored securely in `.env` files
- HTTPS (via Vercel): Ensures secure communication between users and frontend

These measures help protect the system from unauthorized access and potential threats.

2.5 AZURE SERVICES MAPPING AND JUSTIFICATION

AWS Service	Purpose in Project
EC2 (Elastic Compute Cloud)	Used to host and run the backend Node.js server
Security Groups	Used to allow traffic on port 22 (SSH) and port 80 (HTTP)
Public IP Address	Used to connect frontend (Vercel) with backend via internet

Table 6: AWS Service Mapping

CHAPTER 3 - DEVOPS IMPLEMENTATION

3.1 DevOps Practices and Deployment Approach

The FixMate application follows a structured DevOps approach to ensure efficient deployment, reliability, and smooth system operation. DevOps practices help in automating workflows, maintaining consistency between development and production environments, and improving overall system performance. In this project, cloud platforms, version control systems, and process management tools are integrated to create a stable and scalable deployment environment.

The development workflow is managed using Git and GitHub, which enables version control, collaboration, and tracking of code changes. This ensures that updates to the application are organized and can be deployed systematically. The deployment process is designed to be consistent and repeatable, reducing errors and improving maintainability. By leveraging cloud-based services, the application can be accessed globally and can handle real-world usage efficiently.

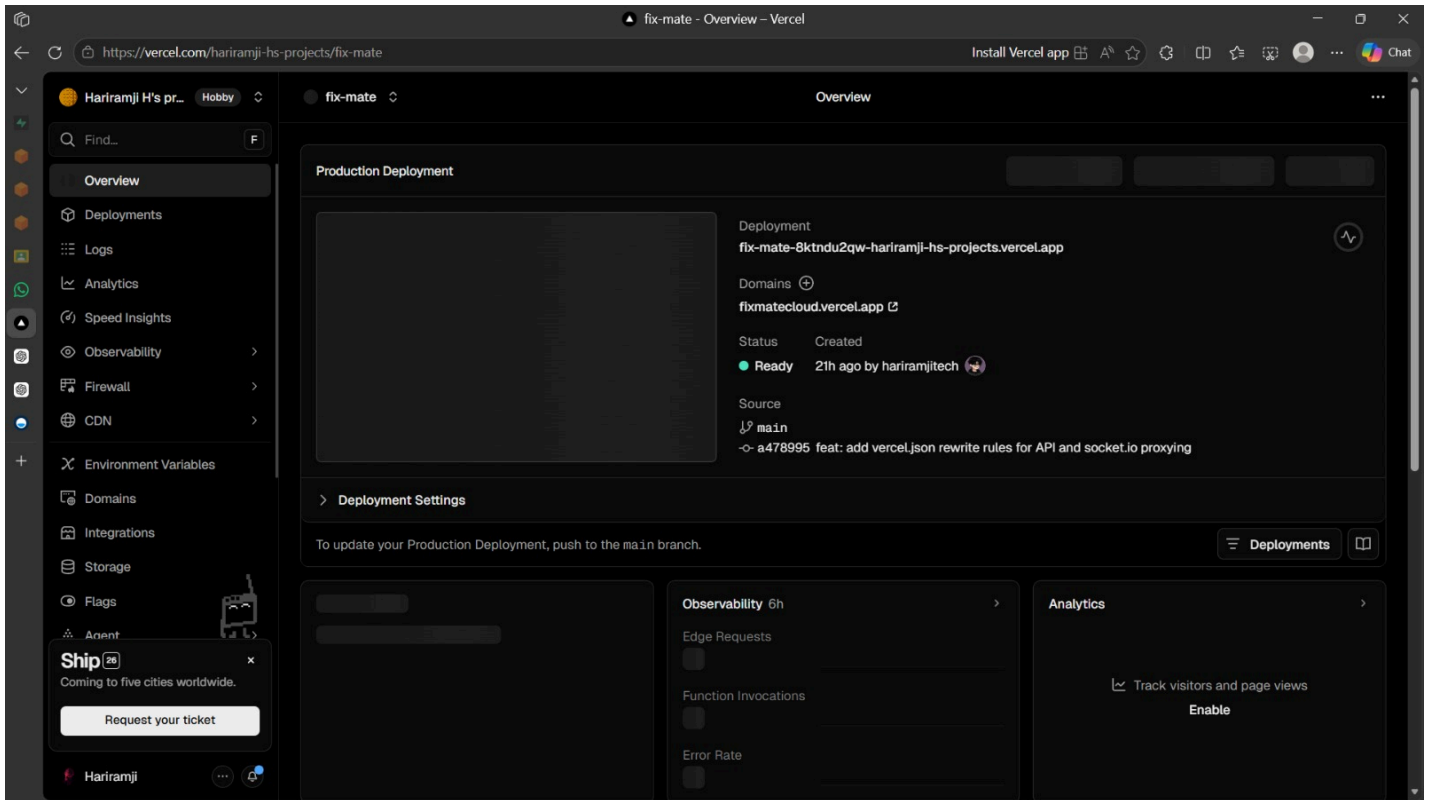
The overall deployment strategy focuses on separating the frontend and backend systems while ensuring seamless communication between them through API calls. This modular approach improves scalability and allows independent updates to different parts of the system without affecting the entire application.

Frontend Deployment

The frontend of the FixMate application is deployed using Vercel, which provides an automated and efficient deployment environment for modern web applications. Vercel is integrated with GitHub, enabling automatic deployment whenever changes are pushed to the repository. This eliminates the need for manual deployment steps and ensures that the latest version of the application is always available to users.

The platform also uses a global Content Delivery Network (CDN), which enhances performance by delivering the application from servers closest to the user. This reduces loading time and improves the overall user experience. The frontend is configured to communicate with the backend through API endpoints, ensuring smooth interaction between the user interface and server-side logic.

This deployment approach ensures that the frontend remains highly responsive, scalable, and easy to maintain, making it suitable for handling multiple users simultaneously.



Backend Deployment

The backend of the FixMate application is deployed on an Amazon Web Services EC2 instance, which provides a flexible and scalable cloud environment. The server runs on Ubuntu and is configured with Node.js to execute backend services. The application code is retrieved from a GitHub repository, and all dependencies are installed using npm.

To ensure continuous operation and reliability, PM2 is used as a process manager. It runs the backend application in the background, automatically restarts it in case of failures, and ensures that the server resumes operation after system reboots. This significantly reduces downtime and ensures uninterrupted service availability.

The backend is also configured with proper environment variables to securely manage sensitive data such as database credentials and API keys. Networking is handled through port configuration and forwarding, allowing the backend to communicate effectively with the frontend.

This setup provides a stable and scalable backend infrastructure capable of handling core functionalities such as user management, service booking, worker coordination, and payment processing.

aws

Search

[Alt+S]

United States (N. Virginia)

Hariramji H (225619513172)

Hariramji H

Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-1004-aws x86_64)

* Documentation: <https://docs.ubuntu.com>

* Management: <https://landscape.canonical.com>

* Support: <https://ubuntu.com/pro>

System information as of Mon May 4 14:55:08 UTC 2026

System load: 0.16

Temperature: -273.1 C

Usage of /: 39.5% of 6.61GB

Processes: 118

Memory usage: 38%

Users logged in: 0

Swap usage: 0%

IPv4 address for ens5: 172.31.41.108

* Ubuntu Pro delivers the most comprehensive open source security and compliance features.

<https://ubuntu.com/aws/pro>

Expanded Security Maintenance for Applications is not enabled.

3 updates can be applied immediately.

To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.

See <https://ubuntu.com/esm> or run: sudo pro status

Last login: Mon May 4 04:27:55 2026 from 18.206.107.28

ubuntu@ip-172-31-41-108:~\$ ^C

ubuntu@ip-172-31-41-108:~\$ pm2 list

id	name	namespace	version	mode	pid	uptime	⓪	status	cpu	mem	user	watching
0	fix-backend	default	1.0.0	Fork	1656	22h	0	online	0%	97.6mb	ubuntu	disabled

ubuntu@ip-172-31-41-108:~\$

i-0543cd8d2bcad2458 (fix-backend)

PublicIPs: 52.91.155.216 PrivateIPs: 172.31.41.108

CloudShell

Feedback

Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

EC2

Instances

Dashboard

AWS Global View

Events

Instances

Instance Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

Capacity Manager

Instances (1/1) Info

Last updated about 9 hours ago

Connect

Instance state

Actions

Launch instances

Saved filter sets

Choose filter set

Find Instance by attribute or tag (case-sensitive)

< 1 >

☒

Name

☒

fix-backend

☒

i-0543cd8d2bcad2458

☒

Running

☒

t3.micro

☒

3/3 checks passed

☒

View alarms

i-0543cd8d2bcad2458 (fix-backend)

Instance ID

i-0543cd8d2bcad2458

IPv6 address

-

Public IPv4 address

52.91.155.216 | open address

Instance state

Running

Private IPv4 addresses

172.31.41.108

Public DNS

ec2-52-91-155-216.compute-1.amazonaws.com | open address

CloudShell

Feedback

Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

CHAPTER 5 - RESULTS AND DISCUSSION

5.1 IMPLEMENTATION SUMMARY

The implementation of the FixMate platform was carried out in a structured and modular manner, ensuring that each component of the system was developed, tested, and integrated efficiently. The development process followed a step-by-step approach, beginning with requirement analysis, followed by system design, frontend and backend development, database integration, and finally deployment.

The frontend of the application was developed using HTML, CSS, and JavaScript, focusing on creating a responsive and user-friendly interface. Special attention was given to usability, ensuring that users could easily navigate through service categories, view details, and book services without confusion. Forms were implemented for user registration, login, and service booking, with validation mechanisms to ensure correct data input.

The backend was implemented using Node.js and Express.js, which handled the core business logic of the application. RESTful APIs were created to manage communication between the frontend and backend. These APIs handled operations such as user authentication, service listing, booking management, and data retrieval. Proper routing and middleware were used to ensure efficient request handling.

The database was integrated using a system such as MongoDB or MySQL, depending on the project setup. It stored essential data including user profiles, service provider details, and booking records. Data was structured in a way that allowed efficient retrieval and updates.

Version control was maintained using Git and GitHub, enabling smooth collaboration among team members. Regular commits ensured that progress was tracked and changes were documented. Deployment was carried out using a cloud platform such as Microsoft Azure, making the application accessible online.

5.2 CHALLENGES FACED AND RESOLUTIONS

During the development of the FixMate project, several challenges were encountered across different stages of implementation. These challenges provided valuable learning experiences and contributed to the overall improvement of the system.

One of the primary challenges was integrating the frontend with the backend. Initially, there were issues with API connectivity, where frontend actions did not trigger backend responses correctly. This was resolved by carefully debugging API routes, ensuring proper endpoint configuration, and verifying request-response handling.

Another significant challenge was managing user authentication and session handling. Ensuring secure

login functionality required proper validation and error handling. This was addressed by implementing structured authentication logic and testing various edge cases such as invalid credentials and unauthorized access.

Database integration also posed difficulties, particularly in designing schemas and ensuring efficient data storage. Issues such as incorrect data mapping and retrieval errors were encountered. These were resolved by refining the database structure and performing thorough testing of queries.

Deployment challenges were also faced, especially when the frontend worked correctly but backend functionalities failed after deployment. This was due to configuration issues and environment mismatches. The problem was resolved by correctly setting environment variables, fixing API base URLs, and ensuring proper server configuration.

5.3 PERFORMANCE OR COST OBSERVATION

The FixMate platform was designed with performance and cost efficiency in mind. During development and testing, several observations were made regarding system performance and resource utilization.

In terms of performance, the application demonstrated efficient response times for most operations, including user login, service browsing, and booking requests. The use of lightweight frameworks such as Express.js contributed to faster backend processing. However, minor delays were observed when handling multiple requests simultaneously, indicating the need for further optimization in future versions.

Database performance was generally satisfactory, with quick data retrieval and updates. Proper indexing and structured queries helped improve efficiency. However, as the volume of data increases, additional optimization techniques such as caching or database scaling may be required.

From a cost perspective, the project was developed using mostly open-source technologies, which significantly reduced development expenses. Tools such as Node.js, Express.js, and GitHub are freely available, making the project cost-effective.

Deployment on cloud platforms such as Microsoft Azure introduced some cost considerations. While basic hosting can be managed within free tiers, scaling the application for real-world usage may involve additional costs related to server resources, storage, and bandwidth.

Overall, the system achieves a good balance between performance and cost. With further optimization and efficient resource management, the platform can be scaled without excessive expenditure.

5.4 KEY LEARNINGS AND TEAM CONTRIBUTIONS

The development of the FixMate project provided valuable technical and collaborative learning experiences for the entire team. Each phase of the project contributed to enhancing both individual skills and teamwork capabilities.

From a technical perspective, the team gained hands-on experience in full-stack development. Members learned how to design and implement user interfaces, develop backend APIs, and integrate databases. The project also improved understanding of client-server architecture and RESTful API communication.

One of the key learnings was the importance of proper planning and requirement analysis. Clearly defining system requirements helped avoid confusion during development and ensured that the project stayed aligned with its objectives.

The team also learned the significance of debugging and problem-solving. Encountering and resolving errors in API integration, database queries, and deployment strengthened analytical thinking and technical expertise.

Version control using Git and GitHub played a crucial role in team collaboration. Members learned how to manage code efficiently, resolve merge conflicts, and maintain a clean project history.

In terms of team contributions, responsibilities were divided among members based on their strengths. Some focused on frontend development, others on backend logic, while a few handled database management and deployment. Regular communication and coordination ensured that all components were integrated smoothly.

CHAPTER 6 - CONCLUSION AND FUTURE WORKS

6.1 CONCLUSION

The FixMate project successfully addresses a significant real-world problem by providing a centralized digital platform for accessing essential everyday services. In traditional scenarios, users often face difficulties in finding reliable service providers due to fragmented sources, lack of transparency, and inefficient communication methods. This project effectively bridges that gap by integrating multiple services into a single, user-friendly system.

Through the implementation of a client-server architecture, the platform demonstrates the practical application of full-stack development concepts. The frontend ensures an intuitive and responsive user interface, while the backend efficiently handles business logic, data processing, and communication with the database. The use of RESTful APIs enables smooth interaction between system components, ensuring reliability and scalability.

The system allows users to register, browse services, and book service providers with ease, thereby enhancing convenience and reducing the time required to access essential services. At the same time, service providers benefit from increased visibility and a structured platform to manage their offerings and interact with customers. This creates a mutually beneficial ecosystem that supports both demand and supply.

Despite encountering challenges such as API integration issues, database management complexities, and deployment difficulties, the project team successfully resolved these problems through systematic debugging, collaboration, and iterative improvements. These experiences contributed significantly to the technical and practical learning outcomes of the project.

Overall, FixMate achieves its core objectives by improving service accessibility, enhancing user experience, and demonstrating a scalable solution that can be extended further. The project serves as a strong foundation for future developments and highlights the importance of leveraging technology to solve everyday problems efficiently.

6.2 FUTURE WORKS

The FixMate platform can be further enhanced through the following improvements and extensions:

- **Online Payment Integration**
 - Implement secure in-app payment systems
 - Support multiple payment methods such as UPI, credit/debit cards, and digital wallets
- **Real-Time GPS Tracking**
 - Enable users to track service providers' location
 - Improve transparency and estimated arrival accuracy
- **Rating and Review System**
 - Allow users to provide feedback on services
 - Help maintain quality and build trust among users
- **AI-Based Service Recommendations**
 - Suggest services based on user preferences and history

- Improve personalization and user engagement
- **Expansion of Service Categories**
 - Include more services such as carpentry, painting, and pest control
 - Increase platform usability and coverage
- **Multi-Location Support**
 - Extend services to multiple cities and regions
 - Scale the platform for a larger user base
- **Mobile Application Development**
 - Develop dedicated apps for Android and iOS
 - Improve accessibility and user experience
- **Advanced Search and Filters**
 - Enable filtering by price, rating, availability, and location
 - Enhance ease of finding suitable services
- **Improved Security Features**
 - Implement stronger authentication and data encryption
 - Ensure user data privacy and protection
- **Admin Dashboard Enhancement**
 - Provide analytics and monitoring tools
 - Improve system management and decision-making
- **Notification System Improvement**
 - Real-time alerts for bookings, updates, and reminders
 - Use push notifications and SMS/email integration
- **Cloud Scaling and Optimization**
 - Optimize performance for handling large user traffic
 - Improve system reliability and uptime